AUTOPRIME CENTRO AUTOMOTIVO

Entrega técnica — API de gerenciamento de oficina

Projeto	API Oficina Mecânica — Tech Challenge
Autor	Gabriel Pereira Costa
Repositório	https://github.com/GabrieIn91Costa/-api-oficina-mecanica-solid
Ambiente	Docker + Kubernetes local (Kind)

Material de apoio para submissão no portal

1. Contexto e objetivo

A AutoPrime Centro Automotivo precisava organizar o fluxo de atendimento, reduzir controles manuais e melhorar a rastreabilidade das ordens de serviço. A solução proposta é uma API REST para centralizar cadastros, abertura e acompanhamento de ordens, elaboração e aprovação de orçamentos e consulta de informações operacionais.

2. Arquitetura da solução

A aplicação utiliza FastAPI e PostgreSQL. A execução containerizada é orquestrada por Kubernetes em um cluster local Kind, no namespace oficina.

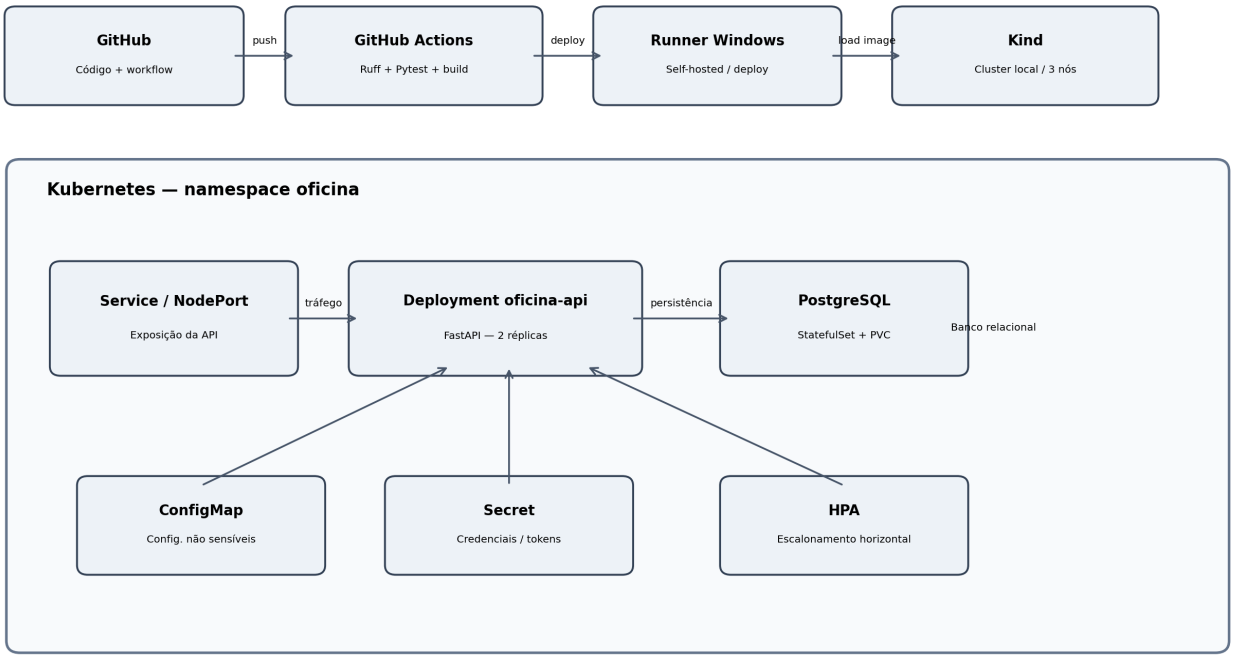


Figura 1 — Visão lógica dos componentes e do fluxo de CI/CD.

3. Componentes de infraestrutura

Componente	Responsabilidade
Docker	Empacotamento da API.
GitHub Actions	Lint, testes, build/publicação e deploy.
GHCR	Registro de imagens Docker.
Runner self-hosted	Executa deploy no ambiente local.
Kind/Kubernetes	Cluster local com três nós.
Deployment	Duas réplicas da API.
PostgreSQL StatefulSet + PVC	Banco relacional e persistência.
Service/NodePort	Exposição da API.
ConfigMap e Secret	Configurações e dados sensíveis.
HPA	Configuração de escalabilidade horizontal.

4. Fluxo CI/CD

O workflow CI-CD Oficina executa Ruff e Pytest. Depois constrói e publica a imagem no GHCR. No deploy da branch main, o runner self-hosted valida Docker, Kind e Kubernetes, carrega a imagem no cluster, aplica os manifestos, aguarda o rollout e executa o health check. O histórico técnico registra execução bem-sucedida do workflow (run #12).

5. Execução e validação

Repositório: https://github.com/Gabrieln91Costa/-api-oficina-mecanica-solid

Cluster local: contexto kind-oficina-local; namespace oficina. Os manifestos Kubernetes estão em k8s/. O Secret oficina-secret deve ser criado previamente com as chaves exigidas pela aplicação. Não inclua credenciais no Git.

Os arquivos Terraform estão em infra/. Antes de provisionar, configure as variáveis e revise o plano. Comandos de validação: terraform init, terraform validate e terraform plan.

6. Materiais e referências

DDD / Event Storming: https://miro.com/app/board/uXjVHsnlPhY=/

Vídeos: diretório videos/ na raiz do repositório. Para o portal, inserir o link público ou não listado do vídeo demonstrativo (máximo 15 minutos) após sua publicação.

Documentação complementar: diretório docs/.

7. Itens a confirmar antes da submissão

• Link público/não listado do vídeo.
• Evidência do HPA escalando sob carga e métricas disponíveis.
• Validação do Terraform.
• Conferência dos endpoints e da coleção da API.
• Confirmação de acesso ao repositório para a equipe solicitada.

Documento elaborado com base no cenário fornecido e no histórico técnico do projeto. Os itens acima são pontos de validação, não afirmações de funcionalidades já comprovadas.